

Online Evolutionary Neural Architecture Search for Multivariate Non-Stationary Time Series Forecasting

Zimeng Lyu*, Alexander Ororbia, Travis Desell

Rochester Institute of Technology, Rochester, NY, USA

Abstract

Time series forecasting (TSF) is one of the most important tasks in data science. TSF models are usually pre-trained with historical data and then applied on future unseen datapoints. However, real-world time series data is usually non-stationary and models trained offline usually face problems from data drift. Models trained and designed in an offline fashion can not quickly adapt to changes quickly or be deployed in real-time. To address these issues, this work presents the Online NeuroEvolution-based Neural Architecture Search (ONE-NAS) algorithm, which is a novel neural architecture search method capable of automatically designing and dynamically training recurrent neural networks (RNNs) for online forecasting tasks. Without any pre-training, ONE-NAS utilizes populations of RNNs that are continuously

*This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Combustion Systems under Award Number #FE00031750 and #FE0031547. It is also supported by the Federal Aviation Administration and MITRE Corporation under the National General Aviation Flight Information Database (NGAFID) award. This work has been partially supported by the National Science Foundation under Grant Number 2225354. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

*Corresponding author

Preprint submitted to Elsevier

April 30, 2023

updated with new network structures and weights in response to new multivariate input data. ONE-NAS is tested on real-world, large-scale multivariate wind turbine data as well as the univariate Dow Jones Industrial Average (DJIA) dataset. Results demonstrate that ONE-NAS outperforms traditional statistical time series forecasting methods, including online linear regression, fixed long short-term memory (LSTM) and gated recurrent unit (GRU) models trained online, as well as state-of-the-art, online ARIMA strategies. Additionally, results show that utilizing multiple populations of RNNs which are periodically repopulated provide significant performance improvements, allowing this successful online neural network architecture design and training.

Keywords: Time Series Forecasting, Online Learning, Neural Architecture Search, Recurrent Neural Networks, Neuroevolution

1. Introduction

Time series forecasting (TSF) is commonly used in many domains, such as health care [1], transportation [2], finance [3], and power systems [4]. TSF models are usually trained offline with historical data, based on the assumption that the data is stationary and data distribution does not change over time. However the underlying data distribution often changes over time in real-world time series data, making it crucial to regularly update the trained models to ensure valid and accurate predictions [5]. Some applications rely on auxiliary methods to transform the data’s non-stationary distribution to a stationary one by training the models in a batch manner (by periodically updating the model with new data) to maintain expected prediction accu-

racy. Unfortunately, this in turn can make models susceptible to catastrophic forgetting if the wrong historical data is removed in future training runs. For offline batch training, models can not adapt to changes in input data and be deployed in real-time to handle rapidly changing scenarios [6, 7].

Online time series forecasting models offer several advantages over traditional batch-based approaches. First, they are designed to adapt to new data points as they arrive, making them more responsive to changes in data distribution. This adaptive nature enables the models to provide more accurate and timely predictions, which can be crucial in fast-paced decision-making processes. Second, online models reduce the need for human intervention, as they automatically update themselves in response to new data, eliminating the time-consuming process of manual model retraining.

Online time series forecasting also brings challenges. The first challenge is catastrophic forgetting – when there is data drift in online TSF data, models tends to forget historical information [8, 9]. Even though rehearsal techniques can help to avoid catastrophic forgetting [10], to ensure online methods can handle real time data, trade offs need to be made between model complexity, learning capability, and efficiency. Additionally, real-world time series data tends to be multivariate, which adds more complexity due to a higher dimension, as well as non-seasonal. This makes it challenging to effectively use classical statistical methods such as the autoregressive moving average (ARMA) [11] and autoregressive integrated moving average (ARIMA) [12] for online multivariate time series forecasting. Lastly, real-world, large-scale online data is usually noisy, and sometimes has incomplete data (*e.g.*, when sensors may not be available), and TSF models need to be robust enough

to deal with incomplete data streams. Models in this domain should also be able to adapt to multiple applications [8].

A major limitation for traditional and current online TSF models is that they assume a single model or neural network architecture which remains fixed, even as it is retrained on new batches of data or incoming online data. Due to this, a model for online TSF will either need to be large enough to be able to capture all possible aspects of concept drift (if data representing those aspects is even available), which is computationally inefficient/intractable, or suffer from catastrophic forgetting when it is trained on new data, limiting its ability to predict data similar to historical observations. In order to overcome these limitations, based on our previous work with time series forecasting on real-world TSF data with neuroevolution (NE) [13, 14, 4], we have developed the Online NeuroEvolution-based Neural Architecture Search (ONE-NAS) algorithm. ONE-NAS, to the authors’ knowledge, is the first algorithm capable of training and evolving recurrent neural networks (RNNs) architectures in an online manner for TSF. Models architectures evolved through ONE-NAS can be quickly trained in response to incoming data streams, which stands in contrast to alternative methods that require significant offline training time using previously gathered sets of training data. Furthermore, ONE-NAS dynamically adapts architectures to keep computational costs minimal while being more robust to catastrophic forgetting.

The novel contributions of our proposed ONE-NAS framework include:

- It is the first online NE neural architecture search (NAS) algorithm for TSF with RNNs,

- It utilizes distributed islands that allow evolution in small niches to preserve diversity,
- It utilizes island repopulation to improve performance over traditional methods,
- It generates genomes (RNN architectures and weights) using a Lamarckian weight initialization strategy that allows information retention (significantly speeding up training),
- It trains genomes on randomly selected historical data, such that important historical data is naturally preserved in the training process,
- It performs well on univariate as well as multivariate datasets,
- It is distributed and scalable, and,
- It is robust to real-world, large scale, and noisy data.

Preliminary work [15] only offered a minimal presentation/sketch of the ONE-NAS algorithm, and this work extends this by providing full treatment of the algorithm, including detailed pseudo-code. Preliminary, earlier work further only compared the performance between ONE-NAS and classical naive, moving average, and exponential smoothing methods, focusing on the performance between ONE-NAS and ONE-NAS utilizing a repopulation method. This paper presents additional experiments comparing ONE-NAS with a traditional statistical TSF methods for online linear regression, online trained fixed architecture LSTM and GRU RNNs, as well as modern online ARMIA based methods – crucially highlighting its feasibility as an algorithm that can both design and train RNNs for TSF in online scenarios.

Results were gathered using noisy, real-world multivariate time series data extracted from wind turbine sensors as well as the univariate Dow Jones In-

Online NeuroEvolution Neural Architecture Search (ONE-NAS)

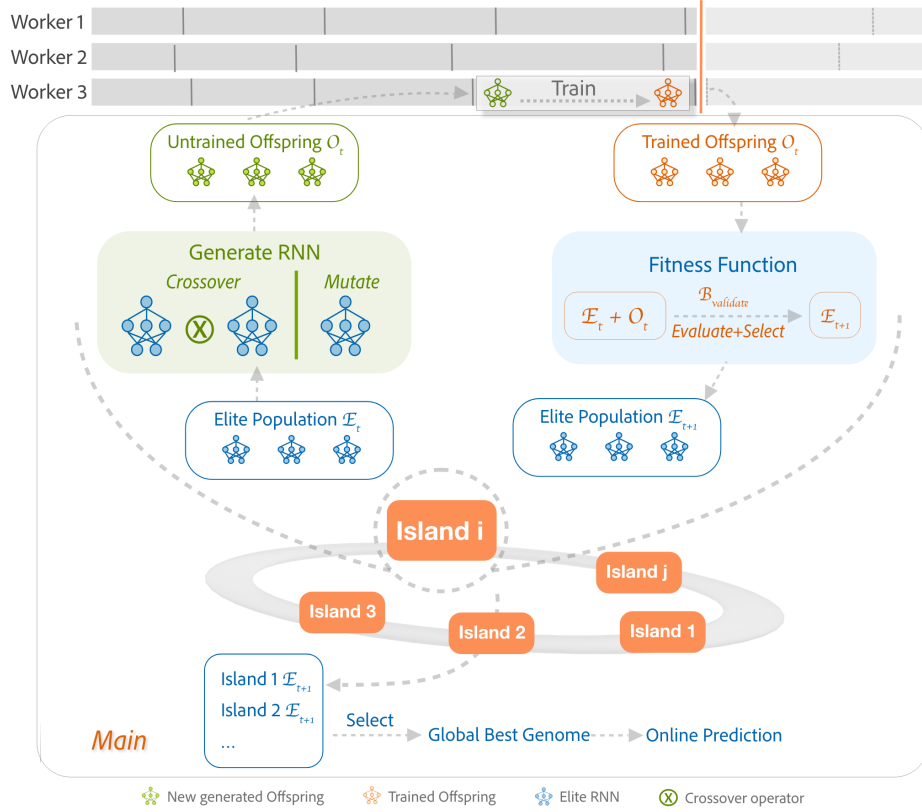


Figure 1: Depicted is one generation of the ONE-NAS online neuroevolutionary process. The global best genome performs predictions concurrently with distributed genome generation and evaluation.

dustrial Average (DJIA) dataset. Our empirical results demonstrate significant improvements in accuracy when using ONE-NAS over these methods. Furthermore, ONE-NAS utilizes a distributed, scalable computational processing scheme and is shown to operate efficiently, in real-time, over short time scales. Furthermore, we show that, within ONE-NAS, utilizing multiple islands that are periodically repopulated to prevent stagnation significantly improves the performance of the underlying optimization process.

2. Related Work

Traditional time series forecasting typically use classical methods on univariate data for simple applications, such as retail sales forecasting [16]. Classical time series forecasting methods include naïve prediction, moving average (MA) prediction [11], autoregression (AR) [17], and exponential smoothing [18] (also called simple exponential smoothing), offering powerful tools for time series forecasting; and traditional statistical methods such as autoregressive moving average (ARMA) [11], autoregressive integrated moving average (ARIMA) models [11], triple exponential smoothing [19] (also known as Holt Winter’s Exponential Smoothing) still play an important role in time series forecasting and have been used in a variety of different datasets [20] [21] [22]. Although researchers explore using classical statistical methods on multivariate datasets, such as multivariate ARIMA [23, 24], the majority of the classical methods only works on univariate dataset, which has a huge limitation in real-world TSF applications.

Classical methods such as MA or exponential smoothing make prediction base on the linear combination of previous observations, therefore they do

not require pre-training. Some classical methods such as AR or ARIMA require pre-training on historical data, and they face the same challenges as other offline learning methods. Recent research has led to the development of online variants of classical models allowing model parameters to adapt to incoming streams of data. The online autoregressive integrated moving average (online ARIMA) has been proposed for online time series forecasting [12], anomaly detection [25], and unsupervised anomaly detection [26] [26]. The autoregressive moving average (ARMA) [27] and the seasonal autoregressive integrated moving average (SARIMA) [28] have also been proposed as powerful models for time series, with Anava *et al.* further proposing an AR model for TSF that can handle missing data values [29].

With respect to the artificial neural network (ANN) based approaches for online TSF, Guo *et al.* proposed an adaptive gradient learning method for training recurrent neural networks (RNNs) capable of time series forecasting notably in the presence of anomalies and change points [30]. Yang *et al.* use RoAdam (Robust Adam) to train long short-term memory (LSTM) RNNs for online time series prediction in the presence of outliers [31]. Wang *et al.* design an online sequential extreme learning machine utilizing a kernel (OS-ELMK) for non-stationary time series forecasting [32]. However all these works focus on TSF with only univariate time series datasets.

TSF models usually face data or concept drift issues, and many works focus on designing TSF models with AutoML to address this [33, 34]. Unfortunately these are not online learning methods. Other than classical or NN-based online algorithms, online neural architecture search (NAS) and AutoML algorithms can also provide methods to address the drift problem

in time series classification and computer vision. Celik *et al.* propose concept drift adaptation strategies for existing AutoML algorithms to automate the ML pipeline, however these works focus on time series classification *et al.* [35] [36]. Madrid *et al.* extend AutoML methods with drift detection techniques for online learning, however they are not applied to TSF [37]. Yan *et al.* proposed a privacy-preserving online AutoML for face detection that extracts the meta-features of the input data with the goal of continuously improving the core algorithm’s performance [38]. To author’s knowledge, there is no existing related work that does online TSF using AutoML or NAS.

Neuroevolution (NE) itself has been widely used for time series prediction and neural architecture search in offline scenarios [39] [14] [13]. However, online neuroevolution has only been rarely investigated, with a few algorithms designed for games or simulators that involve real-time interactions, such as an online car racing simulator [40], online video games [41] [42], and robotic controllers [43]. Crucially, these online NE NAS algorithms are based on the venerable NeuroEvolution of Augmenting Topologies (NEAT) algorithm [44] and start with minimal networks and evolve topologies and weights through a simulated evolutionary process. Cardamone *et al.* developed an online car racing simulator based on NEAT [44] and rtNEAT [42], combined with four evaluation strategies (ϵ -greedy [45], ϵ -greedy-improved, softmax, and interval-based). This algorithm evolves car drivers from scratch and can outperform offline models. To our knowledge, none of the above online neuroevolution algorithms are capable of evolving recurrent networks nor have any been developed to specifically conduct online time series forecasting,

notably making ONE-NAS the first of its kind.

3. EXAMM

This work leverages components from the Evolutionary Exploration of Augmenting Memory Models (EXAMM) algorithm [46] at its core. In particular, our algorithm leverages EXAMM’s mutation, crossover, and training operations while maintaining the population in distinct islands. However, the process by which it evolves artificial neural networks (ANNs) and the operations it provides for utilizing them in online scenarios are novel. This section first describes EXAMM and its leveraged components in detail before describing how they are incorporated into and extended/generalized in ONE-NAS.

Figure 2 provides a high-level flowchart for the EXAMM algorithm. EXAMM is an offline, distributed asynchronous neuro-evolutionary (NE) algorithm that evolves progressively larger RNNs for large-scale, multivariate, real-world TSF [47, 48]. EXAMM has n islands and each island has a capacity of m . Each island starts with a minimum seed genome that only has input-to-output connections. One genome is generated for each island in a round-robin fashion until the entire population reaches the total number of the generated genomes. On each island, the new genome could be generated by mutation, intra-island crossover, or inter-island crossover, in which islands could exchange genes. The generated genome is sent to the next available worker process for training. After training is completed, the trained genome is sent back to the island where it was generated, potentially replacing the worst genome if its fitness is higher than the (current) worst fitness value.

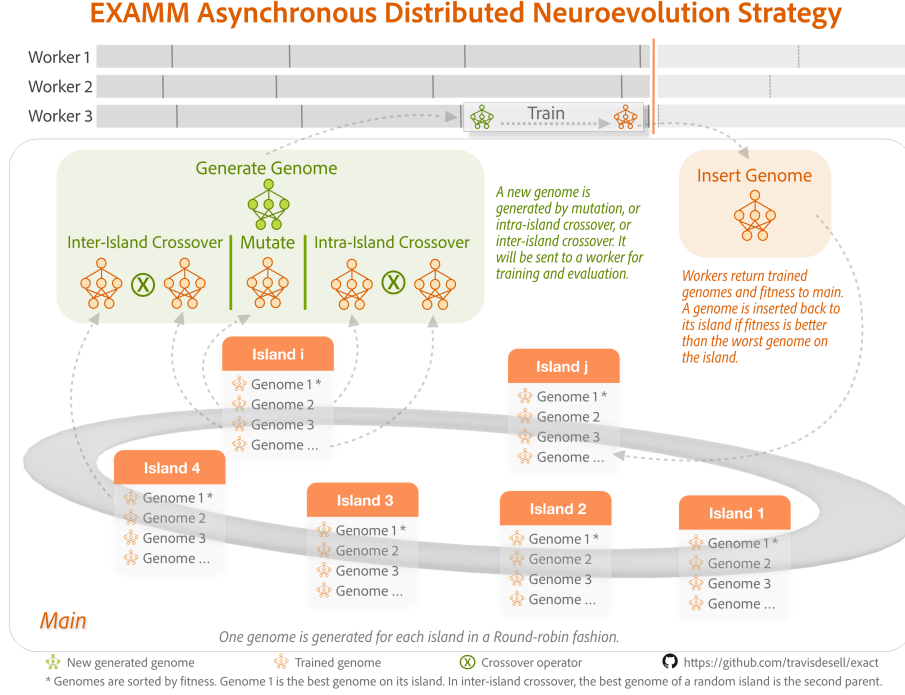


Figure 2: EXAMM flowchart

EXAMM evolves RNN architectures consisting of varying degrees of recurrent connections and memory cells through a series of mutation and crossover (reproduction) operations. Memory cells are selected from a neural library including Δ -RNN units [49], gated recurrent units (GRUs) [50], long short-term memory cells (LSTMs) [51], minimal gated units (MGUs) [52], and update-gate RNN cells (UGRNNs) [53]. ONE-NAS utilizes a similar parallel asynchronous strategy which naturally loads balances itself and allows for the decoupling of population size from the number of workers during each generation [46]. Generated offspring inherit their weights from their par-

ents, which can significantly reduce the time needed for their training and evaluation [13]. It has been shown that EXAMM can swiftly adapt RNNs in transfer learning scenarios, even when the input and output data streams are changed [47] [48]. This serves as a preliminary motivation and justification for being able to adapt and evolve RNNs for online TSF.

EXAMM has also evolved RNNs for time series prediction for different real-world applications [4, 48] and performance improvements through the use of extinction and repopulation events/mechanisms, in real-world evolution [14], have been investigated.

4. ONE-NAS

Algorithms 1, and 2 present pseudo-code for the full ONE-NAS procedure and Figure 1 presents a high-level overview of the asynchronous, distributed, and online ONE-NAS process. ONE-NAS concurrently evolves and trains new RNN candidate models while performing online time series data prediction. *Note that ONE-NAS is fully online and does not require pre-training on any historical data before the online NE process begins.*

Figure 3 presents an example of ONE-NAS online prediction performance on multivariate wind turbine dataset. The plotted value is the expected and predicted output parameter *average active power* with values normalized between [0, 1]. We can see that the output values are non-seasonal.

Algorithm 1 ONE-NAS

```
1: function ONE-NAS MAIN
2:   for  $t$  in generations do
3:      $\triangleright$  Perform predictions concurrently in a new thread,
4:      $\triangleright$  which returns the next time subsequence when complete
5:      $B_{next} = globalBestGenome.onlinePredict()$ 
6:     if islandRepopulation then
7:       if  $t \% extinctFreq == 0$  then
8:          $worstIsland = rankIslands().last()$ 
9:          $repopulate(worstIsland, globalBestGenome)$ 
10:     $\triangleright$  Generate genomes in main process
11:     $O_t = MPIMain.generateGenomes(E_t)$ 
12:     $\triangleright$  Process genomes asynchronously in parallel on worker processes
13:     $B_{validation} = getValidationData(t, numValidationSets)$ 
14:    for genome  $g$  in  $O_t$  do
15:       $\triangleright$  Each worker randomly selects different training data
16:       $B_{train} = getTrainingData(t, numTrainingSets)$ 
17:       $MPIWorker.trainGenomes(g, B_{validation}, B_{train})$ 
18:     $MPIBarrier()$ 
19:     $\triangleright$  Evaluate genomes for next generation
20:     $E_{t+1} = selectElite(E_t, O_t, B_{validation})$ 
21:     $globalBestGenome = getGlobalBestGenome()$ 
22:     $wait(B_{next})$ 
23:     $\triangleright$  Get the latest subsequence of data and add it to the historical data
24:     $timeSeriesSets.append(B_{next})$ 
```

Similarities between EXAMM and ONENAS:. ONE-NAS is also a NE NAS algorithm, so the online learning process typically starts with a minimal seed genome (a minimal seed genomes only has input to output connections

Algorithm 2 Data Selection Methods

```
1: function GETTRAININGDATA( $t$ )
2:   timeSeriesSets[0 :  $t - numValidationSets$ ].shuffle()
3:    $B_{train} = \text{timeSeriesSets}[0 : numTrainSets]$ 
4: function GETVALIDATIONDATA( $t, numValidationSets$ )
5:    $B_{validation} = \text{timeSeriesSets}[t - numValidationSets : t]$ 
6: function GETONLINETESTDATA( $t, numValidationSets$ )
7:    $B_{next} = \text{timeSeriesSets}[t + 1]$ 
```

and contains no hidden layers). It is also possible to start with a previously trained model or generated architecture as the seed genome to bootstrap this process. Genomes are evolved with the same mutation and crossover methods in EXAMM. The population is maintained by distributed islands, which allows islands to evolve in their own niche, with islands exchanging information only by periodic inter-island crossover. Islands in both methods use a repopulation strategy to periodically erase and repopulate the islands that become stuck in local optima [14], and this work shows that this repopulation is critical for achieving viable online performance in ONE-NAS.

Differences between EXAMM and ONE-NAS: EXAMM is an offline algorithm, where the evolutionary process happens offline with each generated genome adapted to the entire offline dataset. In contrast, ONE-NAS collects the online streaming data for online training and evaluation, selecting subsets of historical data to be used for training and validation. In this work, crucially, ONE-NAS operates entirely online without any pretraining¹.

¹ONE-NAS can be seeded with a prior model or topology - however all results in this work start with an untrained, minimal seed genome

At the end of the neuro-evolutionary process, EXAMM selects the global best genome as the optimal solution. However, in ONE-NAS, the genome generation, training, evaluation, and online prediction processes are all performed online and concurrently. ONE-NAS uses the selected best genome from its previously trained population to make online predictions at each time t while a new population of genomes is dynamically trained.

During the online evolutionary process, genomes are generated randomly by crossover and mutation, and inevitably, new generations contain genomes that perform worse than the average population. In the online setting, any algorithm needs to produce on-the-fly predictions with an expected accuracy at real-time, so it cannot afford to have too many poorly performing genomes in the population pool, allowing them to be parents to future offspring. ONE-NAS instead selects an elite population from the generated trained genomes and uses the elite population as online predictor candidates and parents for the next generation.

ONE-NAS’s online NE process starts with a minimal seed genome, which serves as an initial genome. In ONE-NAS each island has two sub-populations, generated and elite².

At time t , ONE-NAS evolves genomes according to the following steps:

- Use the elite population E_{t-1} to generate a set of m genomes through mutation and crossover, defined as the generated population O_t .
- Train the generated genomes O_t using MPI³ workers for a specified

²EXAMM, on the other hand, operates in a steady-state manner so it does not have explicit populations or elite populations.

³The Message Passing Interface [54], the most popular high performance computing

number of iterations/epochs with randomly selected historical data, B_{train} .

- Evaluate all of the current generation’s genomes, E_{t-1} and O_t , using recent validation data, $B_{validation}$, to calculate their fitness values.
- Select the next elite population, E_t , from E_{t-1} and O_t .
- Selecting the global best genome from the elite populations, E_t , from all islands for online prediction.
- Retain the members of the elite population E_t for the next generation.

Each generation lasts for a specified period p , measured in a number of discrete time steps (in this work, $p = 25$), which allows for the processing of a subsequence of the target time series. The best genome from the previous generation performs online predictions of the new subsequence (B_{next}) as it arrives while, concurrently, the new generation of genomes is generated and trained. At the end of a generation, this new subsequence of data is added to ONE-NAS’s historical training data (memory). During a generation, the generated genomes O_t are trained on a randomly selected set of B_{train} subsequences of historical data, after which the entire population (including the elite E_t) will be validated on the most recent $B_{validation}$ subsequences. Each genome’s fitness, calculated as the mean squared error loss over $B_{validation}$, is then used to select the next elite population E_{t+1} . The best genome in E_{t+1} is used for online prediction in the next generation.

Note that, while the genomes O_t are trained using backpropagation on batches sampled from the historical data, the RNNs in E_t do not continue

message passing library.

to be trained. Since not all RNNs in E_t will perform better than those in O_t , “obsolete” RNNs will naturally die off over time while RNNs with strong performance will remain/persist. Also note that one of EXAMM’s mutation operations, crucially utilized by ONE-NAS, is a clone operation which allows for a duplicate of an elite parent to be retained and trained in the next generation (further re-using its parent’s weights due to EXAMM’s Lamarckian weight inheritance strategy).

4.1. Learning Important Information

Online learning models generally suffer from catastrophic forgetting [8] – this poses a significant problem since preserving important historical temporal information is crucial for TSF problems. One way to counter forgetting and data drift would be to train all of the offspring/genomes on every single historical data point seen so far jointly with any new incoming data. However, this would quickly become inefficient, preventing an adaptive system from operating in a fast and online fashion. In ONE-NAS, offspring are instead trained with new incoming data and only on randomly selected sub-sets of historical data. Historical information is preserved in the population by retaining elite individuals as well as children inheriting weights from those parents [13] in tandem with efficient training using only small amounts of randomly selected/stored historical data. The fitness of the trained offspring is evaluated via validation mean squared error (MSE) using the most recent data. This process results in genomes that contain less important temporal information that naturally die off across generations of evolution.

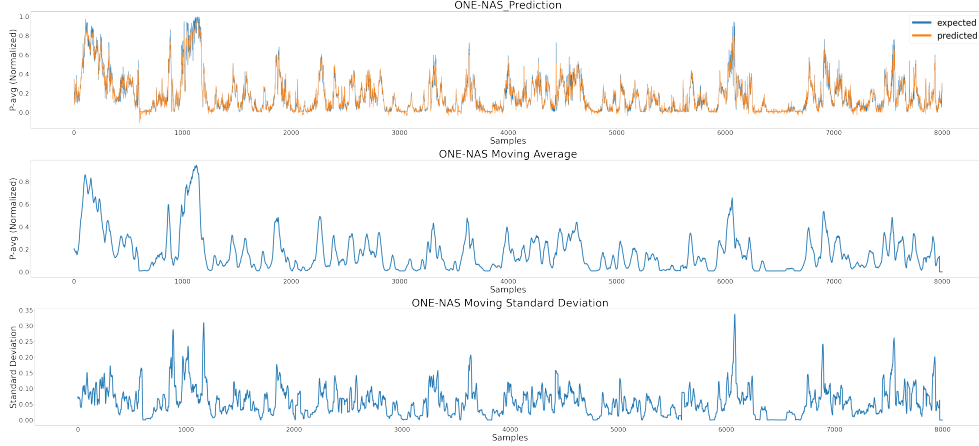


Figure 3: The ONE-NAS prediction performance predicting *active power* (top) of wind turbine engine data, the output parameter moving average (middle) and moving standard deviation (bottom) with window size 25. The output parameter is non-seasonal, very noisy, and contains multiple distribution shifts.

5. Experimental Design

5.1. Datasets

This work utilized two real-world datasets for predicting time series data with RNNs. The first was wind turbine engine data collected and made available by ENGIE’s La Haute Borne open data wind farm⁴, which was gathered from 2017 to January 2018. This wind dataset is very long, multivariate (containing 22 parameters/dimensions), non-seasonal, and the parameter recordings are not independent. The wind turbine data consists of readings every 10 minutes from 2017 to January 2018. The input parameters are:

⁴<https://opendata-renewables.engie.com>

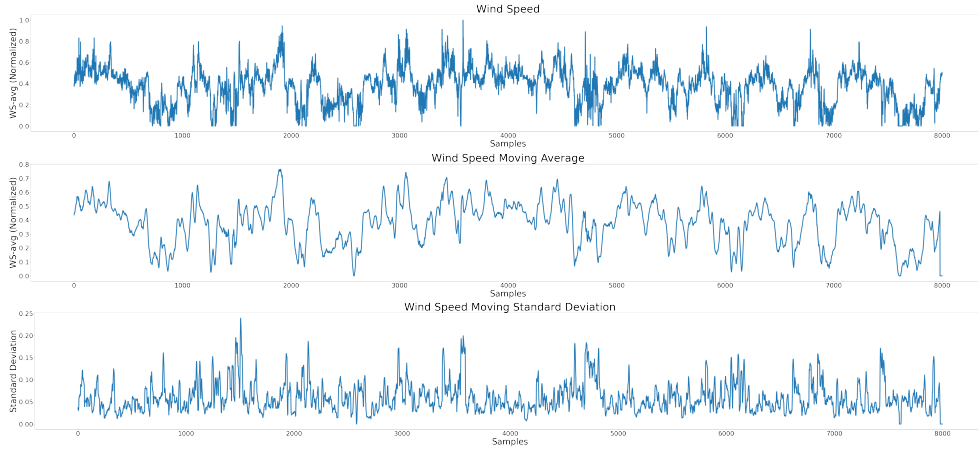


Figure 4: The input parameter *wind speed* of wind turbine engine data (top), the input parameter moving average (middle) and moving standard deviation (bottom) with window size 25, the output parameter online average and standard deviation. The input parameter is non-seasonal, very noisy and has multiple distribution shifts.

- Pitch Angle
- Hub Temperature
- Generator Converter Speed
- Converter Torque
- Active Power
- Apparent Power
- Power Factor
- Generator Bearing 1 Temperature
- Generator Bearing 2 Temperature
- Generator Stator Temperature
- Gearbox Bearing 1 Temperature
- Gearbox Bearing 2 Temperature
- Gearbox Inlet temperature
- Gearbox Oil Sump Temperature

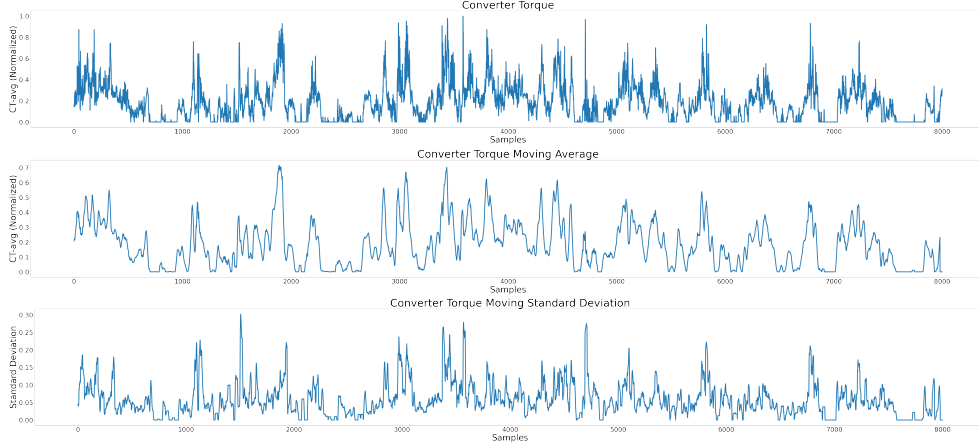


Figure 5: The input parameter *converter torque* of wind turbine engine data (top), the input parameter moving average (middle) and moving standard deviation (bottom) with window size 25, the output parameter online average and standard deviation. The input parameter is non-seasonal, very noisy and has multiple distribution shifts.

- Nacelle Angle
- Outdoor Temperature
- Nacelle Temperature
- Grid Frequency
- Wind Speed
- Grid Voltage
- Absolute Wind Direction
- Rotor Bearing Temperature

Active Power was selected as the output parameter to forecast for the wind turbine dataset.

The top plot in Figure 3 provides an example of the noisiness and complexity of the output parameter as well as an example of the accuracy of ONE-NAS's predictions. This entire time series has around 59,000 data points, with the above example showing 7000 data points from February to April 2017 as an example. Note that ONE-NAS is trained online (with no

pre-training) so this plot depicts online predictive performance as our system learns from scratch. The second dataset is the daily index of the Dow Jones Industrial Average (DJIA) from the years 1885-1962, which is univariate and contains 35,701 samples. Both of the datasets that we investigate contain raw and abnormal data points that have not been cleaned – spikes and outliers have not been removed or smoothed from either dataset.

The middle and the bottom plots in Figure 3 provide the moving average and moving standard deviation of the output parameter. Figure 4 and 5 show two representative parameters from the 22 input parameters and their moving average and moving standard deviation of the wind turbine dataset. The window size for the moving average and moving standard deviation is 25. The two input parameters are *wind speed* and *converter torque*. *Wind speed* is the velocity of the wind at a specific location and time, and the amount of power generated by a wind turbine is directly related to the wind speed. *Converter torque* refers to the torque (rotational force) transmitted through the power converter system of the wind turbine. Both are important input parameters in the wind turbine dataset, and the plots show the data distribution of input and output parameters change over time. This real-world dataset is non-seasonal, very noisy, and as the moving average and standard deviation show, has numerous instances of distribution drift (*e.g.*, when weather changes or turbines are shut down).

5.2. Processing and Setup

Research has shown that using shorter subsequences of time series data during training can improve an RNN’s convergence rate as well as its overall performance [4]. For these experiments, the original datasets were divided

into subsequences of 25 timesteps each. During each simulated ONE-NAS generation, each newly generated genome was trained on 600 randomly selected subsequences from the historical data pool and then validated using the most recent 100 subsequences of data. Each genome utilizes a different random selection of 600 subsequences. There is no overlap between training and validation data (the most recent 100 subsequences are added into the historical pool after being used for validation). All the experiments for the wind dataset were run for 2000 generations, which represents one singular pass over the full wind data time series.

In the ONE-NAS simulations⁵, during each generation, 50 elite genomes from the previous generation were retained and the elite genomes were used to generate 100 new genomes using a mutation rate of 0.4 and crossover probability of 0.6. Each of the 100 non-elite genomes in the new generation were trained in a worker process for 10 epochs of backprop (a local search), with the first 5 epochs involving training on the original subsequence data. Then, in each of the last 5 epochs, 10% Gaussian noise was added (using the mean and standard deviation of the sliced data) as an augmentation technique to prevent overfitting [55] [56]. ONE-NAS with island repopulation utilized 10, 20, 30, or 40 islands, each having its own elite population of 5 genomes which generated an additional 10 genomes per generation. New genomes were generated with a mutation rate of 0.3, inter-island crossover rate of 0.4, and intra-island crossover rate of 0.3.

In the ONE-NAS simulations⁶, during each generation, ONE-NAS with

⁵<https://github.com/travisdesell/exact>

⁶<https://github.com/travisdesell/exact>

island repopulation utilized 10, 20, 30, or 40 islands, each having its own elite population of 5 genomes which generated an additional 10 genomes per generation. New genomes were generated with a mutation rate of 0.3, inter-island crossover rate of 0.4, and intra-island crossover rate of 0.3. Each of the 10 non-elite genomes in the new generation were trained in a worker process for 10 backpropagation epochs, with the first 5 epochs involving training on the original subsequence data. Then, in each of the last 5 epochs, 10% Gaussian noise was added (using the mean and standard deviation of the sliced data) as an augmentation technique to prevent overfitting [55] [56].

The choice of historical, validation pool size, elite and non-elite population sizes, along with the number of back propagation epochs, as well as mutation and crossover rates, were hand tuned through prior experience in running ONE-NAS experiments.

5.3. Results

Each experiment was repeated 10 times using Rochester Institute of Technology’s research computing systems. This system consists of 2304 Intel® Xeon® Gold 6150 CPU 2.70GHz cores and 24 TB RAM, with compute nodes running the RedHat Enterprise Linux 7 system. Each experiment utilized 16 cores.

5.3.1. Comparison with Classical TSF Methods

To test the performance of ONE-NAS, we first compared it to classical TSF methods: naive prediction, moving average prediction, and simple exponential smoothing. While these are very simple univariate methods, they do perform well in many real-world application scenarios, due to the fact that

the TSF data is very noisy and using previous observations to estimate the expected value at next time step usually gives a reasonable prediction. Notably, these methods are capable of outperforming complex and sophisticated methods across a wide variety of datasets [57].

Naïve prediction (Naive) simply uses the data’s/parameter’s previous value, x_{t-1} as the predicted value, $\hat{y}_t$, for the next time step: $\hat{y}_t = x_{t-1}$. The moving average [11] predictor (MA) uses the average of the last n time steps as the prediction of the next time step, where n is the moving average data smoothing window (a hyperparameter): $\hat{y}_t = 1/n * \sum_1^n x_{t-n}$. The simple exponential smoothing (Holt linear) [18] predictor (EXP) computes a running average of the previously seen parameters, where α is the smoothing factor, and $0 < \alpha < 1$: $\hat{y}_t = \alpha * x_{t-1} + (1 - \alpha) * \hat{y}_{t-1}$.

We are aware that the choice of window size and α significantly affect the prediction performance of the moving average and exponential smoothing methods. Figure 6 shows the MA and EXP prediction mean squared error (MSE) with different window sizes (n) and α values on the wind dataset. The plot shows that the wind dataset is highly complex, where naïve almost entirely predicts better than MA or EXP (apart from a negligible improvement with EXP for *alpha* values of 0.8 and 0.9).

Figure 7 shows a plot for the online prediction MSE of ONE-NAS. The horizontal lines show each of the three classical time series forecasting methods. Note that these methods are not stochastic, so their performance is always the same. Moving average (MA) predicts with a window size $n = 3$ and exponential smoothing uses a $\alpha = 0.2$. The difference between “OneNas” and “OneNas Repopulation” on the plots is that “OneNas” does not

use any islands, where the entire population is one island, and the “OneNas Repopulation” uses multiple islands and the island repopulation strategies. The ONE-NAS repopulation strategy shown in this plot used 20 islands (each with 5 elite genomes and 10 other genomes per generation), with an extinction and repopulation frequency of 200 generations. While the ONE-NAS online prediction without repopulation performs worse compared to all three classical methods, the ONE-NAS method with repopulation not only significantly outperforms the baselines but exhibits better predictive performance than the classical methods across all repeats except for one outlier.

We further investigated using linear regression for online learning. At time t , the linear regression model is built using the last n observations and is then used to make predictions for time $t + 1$. Figure 8 shows the average linear regression online prediction mean squared error (MSE) using different observation window sizes n on the wind dataset. The plot shows that, for this dataset, the linear regressor makes better predictions when the observation window n is smaller. The best results come from the window size of 1, which is simply the naïve prediction from before. The inability of these methods to outperform naïve prediction highlights the complexity of this dataset – simpler prediction methodologies cannot capture the complexity of forecasting this data. Due to this, we focus on using naïve prediction MSE to represent classical, baseline prediction performance in the following sections/experiments.

5.3.2. Preserving Population Variety with Islands

To study the effect of using islands on ONE-NAS, we compared the performance of using islands versus using a single large population. In figure 7,

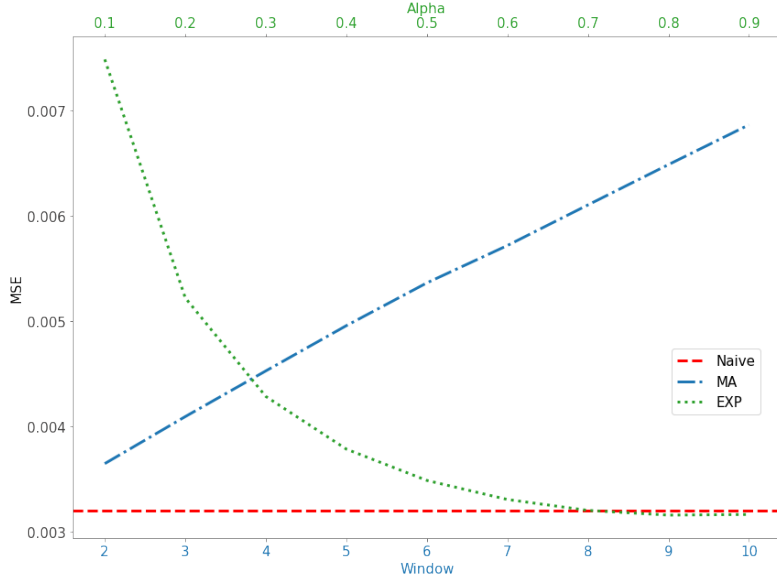


Figure 6: Classical TSF methods with varying window sizes and alpha values.

the “ONE-NAS” group uses only one elite population and one generated population. The “ONE-NAS Repopulation” group utilizes has islands of populations, with each island having an elite population and a generated population. The total population size for both experiments was the same. As shown in Figure 7, using islands to maintain the population can significantly improve/boost the online forecasting performance, allowing ONE-NAS to outperform the classical methods. It is crucial to preserve variety in an online setting, because: **1)** ONE-NAS generates more genomes than EX-AMM (for the same dataset and experimental set up, only 8k genomes were generated using EXAMM, but 200k generated in total using ONE-NAS), **2)** genomes generated online are trained and evaluated with significantly less data (only a subset of the historical data), and, **3)** the elite population is

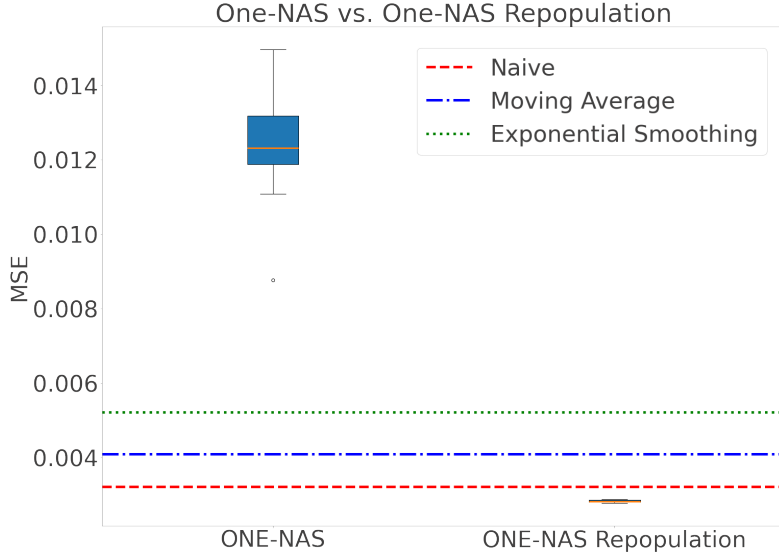


Figure 7: Online prediction MSE of ONE-NAS and three classical TSF methods.

evaluated using the most recent data, so there is some data drift present in each generation, meaning that preserving variety can preserve important historical information while preventing overfitting.

Given the above result that it is possible to effectively evolve and train RNNs in an online setting, we found that two hyperparameters significantly affect online prediction performance in ONE-NAS: the island size and the extinction and repopulation frequency. Figure 9 shows a box plot of the online prediction MSE using island sizes of 20, 30, and 40 in 10 repeated experiments/trials with varying repopulation frequencies. As the number of islands increases, the prediction performance improves. This could be due to the fact that more islands allow for more speciation and a greater ability to escape local optima. Additional species also provide more robustness to noise and overfitting of the data. Given the same number of islands, more frequent

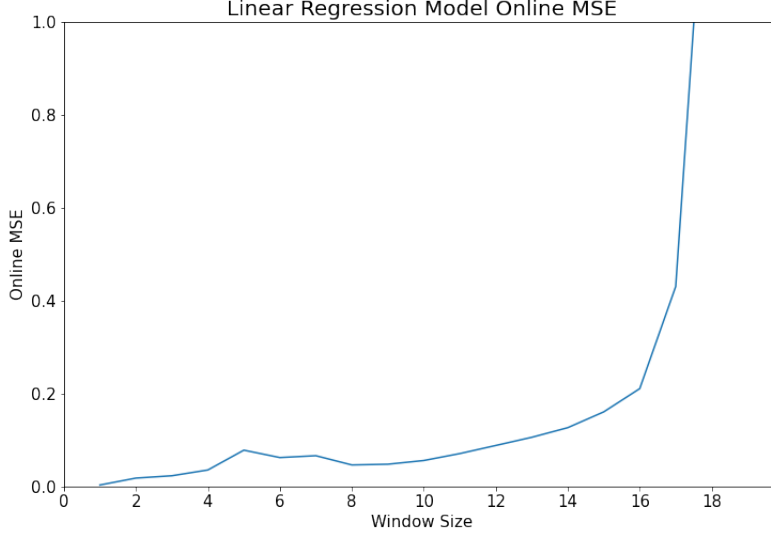


Figure 8: Linear regression window size vs. MSE.

extinction and repopulation events yield better performance on average. This is most likely due to the fact that these events prevent islands from stagnating with poor(er) species.

5.3.3. Online Predictions over Time

The previous plots show the overall performance of ONE-NAS against classical methods for the entire wind time series, which gives an advantage to classical methods since they do not require any training and thus have a significant advantage for earlier time steps when ONE-NAS has not had much opportunity to train/evolve RNNs. In order to investigate how much the RNNs evolved by ONE-NAS were improving as they processed more data in an online fashion, we measured how many time steps (for each generation) exhibited a better predictive performance between the naïve method

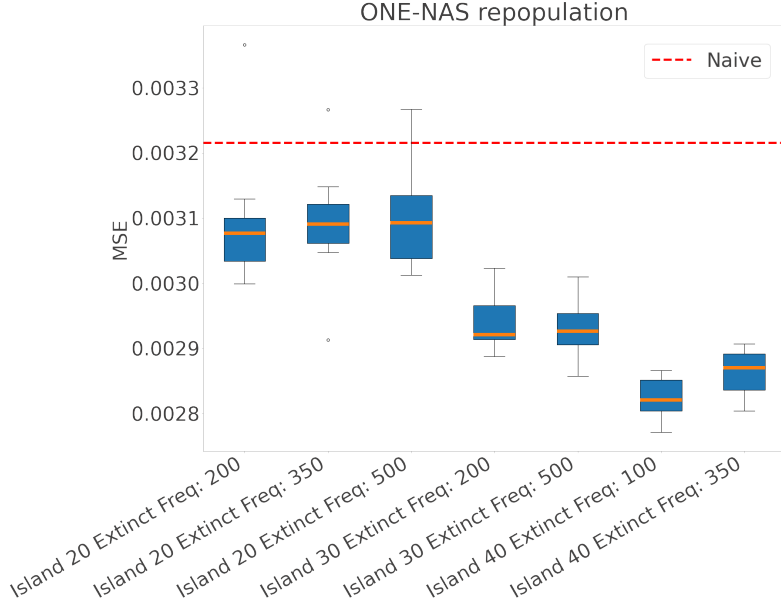


Figure 9: ONE-NAS MSE with varying island sizes and extinction rates.

and ONE-NAS. Figure 10 shows the percentage of predictions of each method that were accurate as the simulation progressed, with the red line representing 50%. This plot shows that, while ONE-NAS does not outperform the naïve strategy within the first 500 generations, the performance of ONE-NAS continues to increase as the evolution continues, which means that ONE-NAS does not only train and predict values online, it also gets progressively better throughout the evolutionary process, which is what we would expect from an online algorithm.

Also note that it would be beneficial to combine a classical method with ONE-NAS – it could prove fruitful to use a naïve predictor until ONE-NAS has had enough evolution time to produce more accurate predictions. Figure 11 shows an example of incorporating naïve predictor to ONE-NAS at

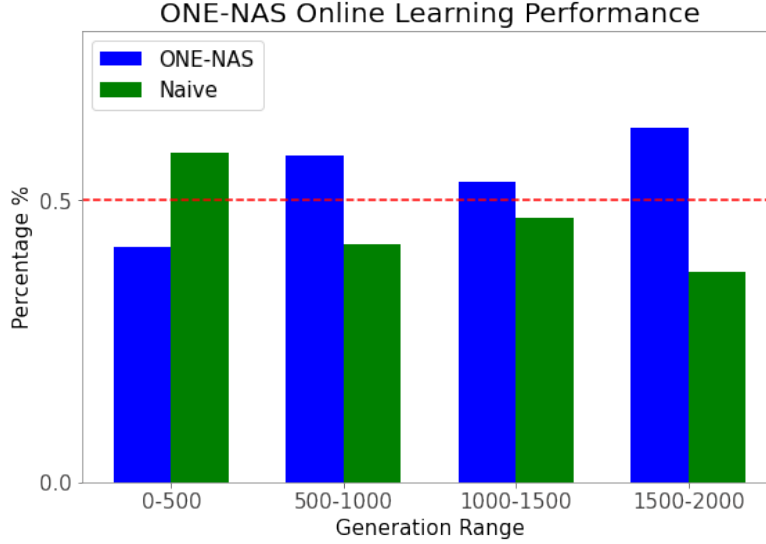


Figure 10: ONE-NAS percentage of better prediction generations versus classical methods.

the beginning of the evolution. The top figure shows the prediction performance of ONE-NAS, the bottom figure shows the prediction performance of incorporating naïve for predicting the first 500 generations (each generation predicts 25 time steps, in total 12500 samples) in ONE-NAS. This simple approach reduced the average prediction MSE from 0.0073 to 0.0032. This provides an easy to implement approach which further increases the performance of ONE-NAS during the beginning of the process.

5.3.4. Online Prediction Time Efficiency

Another key concern for the evaluation of online algorithms, apart from predictive accuracy, is time efficiency. If an online algorithm cannot provide predictions at a rate less than the arrival rate of new data to be predicted, then it is not usable/viable. ONE-NAS resolves this issue by utilizing the

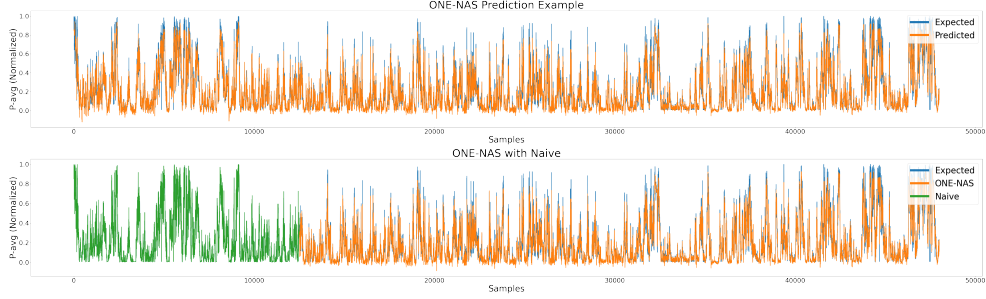


Figure 11: ONE-NAS prediction (Top), ONE-NAS with Naive incorporated (Bottom) on wind turbine dataset

Num Islands	Avg Time (s)	Longest Time (s)
20	35.67	109.20
30	41.72	127.44
40	69.96	189.72

Table 1: The average and longest measured times, in seconds (s), needed to evolve each generation for the wind dataset.

previous best genome to provide predictions while concurrently training the next generation.

Note that for this study, each generation was generated and trained over a single subsequence of 25 time steps. For the wind dataset, each time step was gathered at a 10 minute interval, so this provides a significant buffer. However, for many time series datasets, time step frequency can be per minute, per second, or even faster - making time efficiency a serious concern. Table 1 presents the average and longest time required to evolve and train one generation of the ONE-NAS Repopulation experiments for the varying numbers of islands. Note that population size was tied to island size,

with 5 elite genomes and 10 other genomes per island, which is why the 40 island genomes took approximately twice the time. In the worst case, for 40 islands, the longest time per generation was a bit above 3 minutes, which is far below the 250 minute generation time for the wind data.

It should also be noted that the workers training the genomes for each generation were distributed across 16 processors and that performance will scale linearly upwards until the number of available processors is equal to the population size (i.e., all generated genomes can be independently trained in parallel without reduction in performance apart from a fixed communication and genome generation overhead cost). This particular scalability of ONE-NAS makes it well-suited to online learning. For example, if we scaled up to 200 processors over the 16 used for the 20 island experiments, we can estimate approximately 2.85 seconds per generation (*i.e.*, training the 200 non-elite genomes at once, instead of 16 at a time), plus some additional communication and generation overhead. With a generation time of 25 time steps, this would allow for incoming data to be processed at almost 10 readings per second. Given some flexibility in determining subsequence/generation time, ONE-NAS demonstrates the potential to operate for very high frequency time series given enough computing power.

5.3.5. ONE-NAS vs LSTM & GRU RNNs

LSTM and GRU RNNs have been widely used in TSF problems [58] [59]. Fixed one-layer and two-layer LSTM and GRU networks were trained online to compare with ONE-NAS, where one-layer indicates that an LSTM/GRU only contained one fixed hidden layer or neurons while two-layer indicates that the RNN contained two fixed hidden layers. The size of each hidden

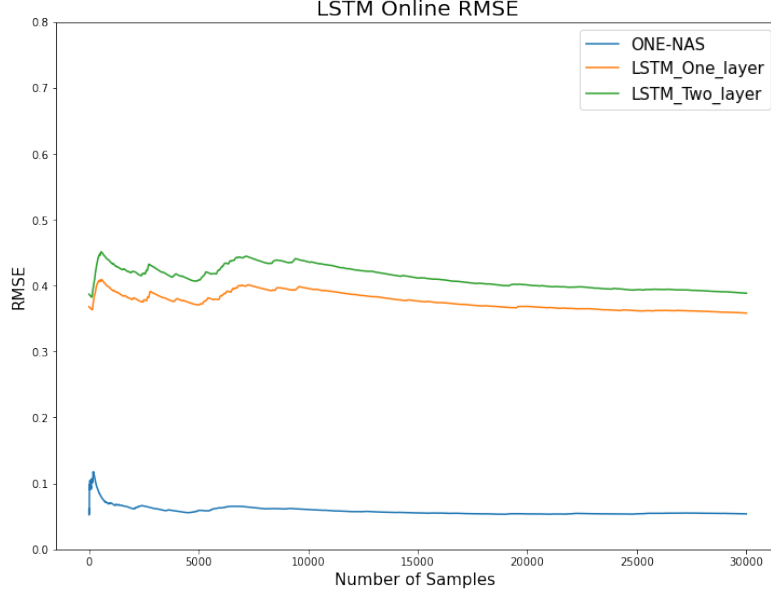


Figure 12: ONE-NAS versus one and two layer online LSTM RNNs.

layer was set to be equal to the size of the input layer and each node/unit in the hidden layer was set to be an LSTM or GRU cell (whereas the input and output layer nodes consisted of simple neurons). Note that each layer is fully connected to the next. The LSTM and GRU networks were initialized with uniform random $\mathcal{U}(-0.5, 0.5)$ weights and weight updates/gradients were applied using Nesterov’s accelerated gradient with a momentum value of $\mu = 0.9$.

The LSTM and GRU networks were trained on the wind data with a fixed moving window size. The gradients for the fixed-layer RNNs were all computed online using truncated back propagation through time [60] [61] (where the networks were unrolled over the length of the window in order to compute the full gradients). Figure 12 and 13 show online testing RMSE

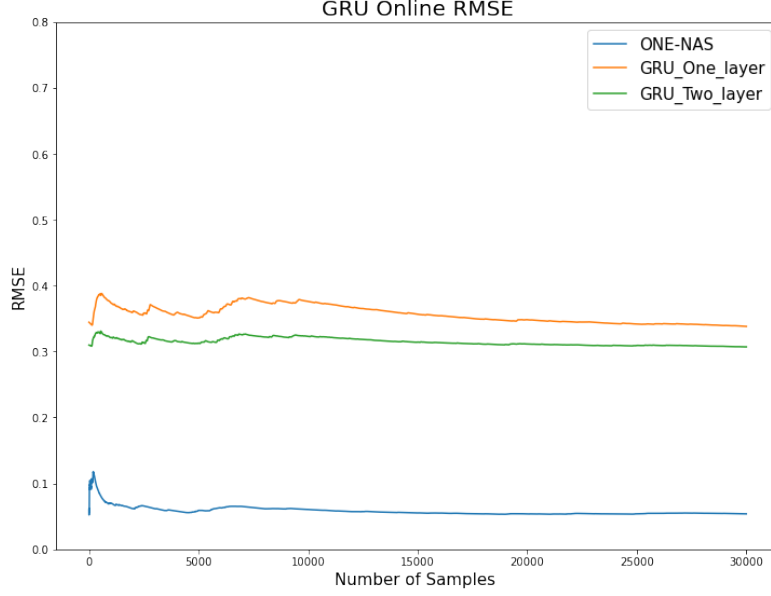


Figure 13: ONE-NAS versus one and two layer online GRU RNNs.

using the best-found window size and best-found learning rate over 10 runs. A window size of 30 and a learning rate of $1e^{-3}$ was used for both one-layer LSTM and GRU networks, a window size of 20 and learning rate of $1e^{-3}$ was used for for the two-layer LSTM network, and a window size of 20 and learning rate of $2e^{-3}$ was used for the two-layer GRU network. These configurations were selected from experiments with different fixed window size of $n = 10, 20, 30, 40, 50$, and a variety of potential learning rates ranging from $5e^{-3}$ to $1e^{-4}$. The results show that ONE-NAS significantly outperforms tuned, fixed one-layer and two-layer LSTM and GRU networks.

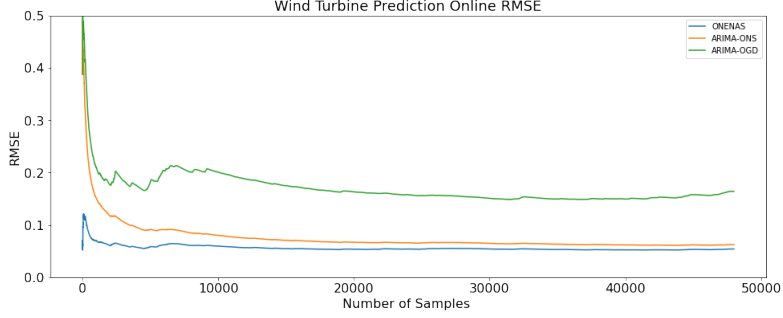


Figure 14: Online ARIMA versus ONE-NAS predictions on the wind dataset.

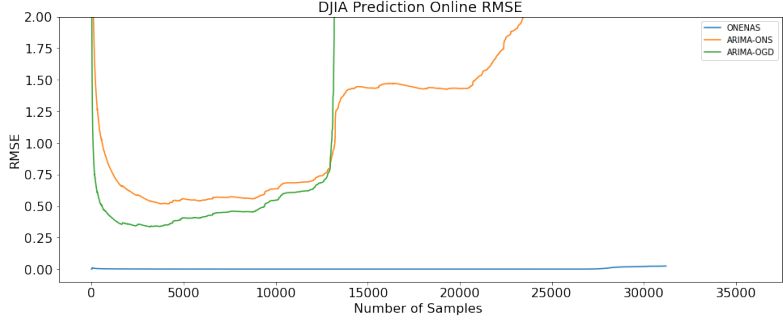


Figure 15: Online ARIMA versus ONE-NAS predictions on the DJIA dataset.

5.3.6. ONE-NAS vs Online ARIMA

While there is a significant lack of methods for online multivariate TSF, recent work by Liu *et al.* [12] has led to the development of an online ARIMA method for univariate TSF. To compare ONE-NAS with a state-of-the-art method as opposed to only the classical methods investigated earlier, we investigate ONE-NAS performance alongside this powerful, online ARIMA model. To reproduce the results from Liu *et al.*, we first performed experiments using the Dow Jones Industrial Dataset (DJIA), which was used for evaluation in their work. Figure 15 presents results for their ARIMA-ONS

(Arima Online Newton Step) and ARIMA-OGD (ARIMA Online Gradient descent) variants, which were the best performing variants examined in the original study. These were compared to the ONE-NAS Repopulation method with 10 islands and extinction frequency of 200, and over a similar generation and subsequence length of 25. For both ONE-NAS and online ARIMA, the plots show that the online root mean squared error (RMSE) over time, averaged over 10 repeated experiments. The online RMSE over time is calculated as the average RSME of all previous predictions. For this DJIA data, we show that, although the online ARIMA predictor mirrors/reproduces the results from the original study, ONE-NAS clearly outperforms this method by multiple orders of magnitude.

We finally compared ONE-NAS with online ARIMA on the wind datasets, the results of which are presented in Figure 14. Similarly, the results depict the average performance over 10 repeated experiments. For online ARIMA, we performed a hyper-parameter sweep using a grid search for the online ARIMA methods on its learning rates and ϵ and report the best found hyper-parameters. For ARIMA-ONS, the learning rate was set to e^{-3} and $\epsilon = 3.16e^{-6}$. For ARIMA-OGD, learning rate was set to e^3 , and $\epsilon = e^{-5.5}$. ONE-NAS used the best hyper-parameters from our earlier previous results/experiments in this study (40 islands and extinction frequency of 100). Similarly, we find that ONE-NAS also significantly outperforms the online ARIMA methods on the wind dataset.

5.4. ONE-NAS Evolved RNNs

Figure 16 presents an example RNN evolved by ONE-NAS. In comparison to commonly used layer-based (hierarchical) neural networks, networks

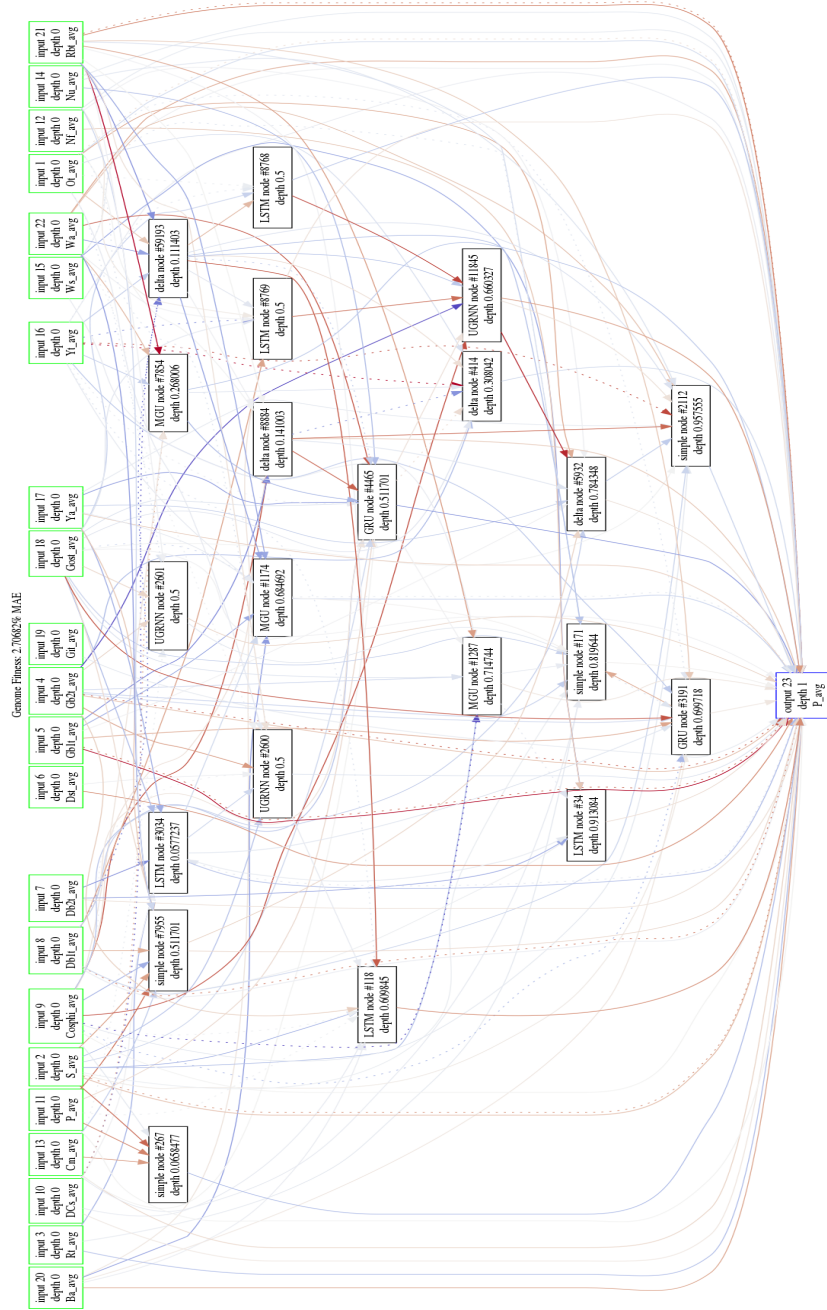


Figure 16: An exemplar best performing RNN that was evolved online by ONE-NAS.

evolved by ONE-NAS are “unlayered” but exhibit highly complex connectivity structures. Nodes in the network show their selected memory cell type (or simple, if a basic neuron was chosen), and edges with positive weights are shown in blue, and edges with negative weights are shown in red. Feed forward connections are in solid lines, and recurrent connections are shown in dotted lines.

While this network may seem complex, in comparison to a standard layer based GRU or LSTM network (which ONE-NAS was shown to outperform in Section 5.3.5), the evolved network only has 21 neurons (of varying types). Whereas the GRU and LSTM networks have hidden layers of size 22 (the same size as the input layer). Furthermore, the nodes in the evolved network are not fully connected and, as a result, actually contains significantly less synaptic edges. This example shows how ONE-NAS not evolves well performing architectures online but also ones that are sparser (with respect to synaptic connectivity) and computationally more efficient.

6. Conclusion

This work presents the Online NeuroEvolution-based Neural Architecture Search (ONE-NAS) meta-heuristic optimization algorithm and applies it to the problem of non-stationary time series forecasting (TSF) on challenging real-world tasks. To the author’s knowledge, ONE-NAS is the first neural architecture search algorithm capable of designing and training recurrent neural networks (RNNs) in real-time as data arrives in an online fashion. ONE-NAS is a dynamic/online, distributed, scalable, real-time algorithm that works on univariate and multivariate real-world TSF datasets.

ONE-NAS is an online NE NAS algorithm which starts evolution from a minimal seed genome, and progressively evolves more complex genomes (neural networks). The process of genome reproduction and selection by fitness evaluation allows it to reduce optimal model complexity [44], and can make it robust to real-world time series data from different domains and at different scales. ONE-NAS collects new streaming data into a historical data pool and new generated genomes are trained on randomly selected sub-sets of the historical data. Generated genomes retain knowledge from parental weights using a Lamarckian inheritance process [13], reducing the amount of training required. By training new genomes with randomly selected historical data and evaluating these on recently collected data, data drift can be managed and catastrophic forgetting can be avoided, as is shown in results on challenging data with these issues. Maintaining genomes in populations also acts as a method to retain previously gained knowledge on historical data to further safeguard against forgetting. An important feature of the ONE-NAS algorithm is that it utilizes islands to maintain the population diversity and prevent over-fitting during the online learning process. Further, periodically repopulating poorly performing islands was shown to be critical in allowing ONE-NAS to outperform other strategies

Our results show that ONE-NAS outperforms other online strategies, such as classical TSF methods, linear regression models, LSTM and GRU networks trained online, and a modern online ARIMA method. Our results also demonstrate that using more islands with more frequent repopulation is strongly correlated with increasing/improved performance. In terms of its adaptability, it can adapt to both low-dimensional (DJIA, univariate) and

high-dimensional (wind turbine engine, multivariate) data, as well as quickly adapt to the data distribution shifts over time. In terms of the algorithm performance, our statistical results indicate that our algorithm can achieve real-time performance in these real-world scenarios.

As a result, this study shows that online neuroevolution or neural architecture search is feasible in online scenarios, which holds great promise for addressing important challenges in time series forecasting. Future work involve dynamically select and retain historical data to reduce memory usage. And further investigation is needed to incorporate classical methods into early evolutionary stage of ONE-NAS for better performance. We further plan to investigate ONE-NAS on other datasets, as well as compare it to other online fixed architecture RNN strategies [30] [31] [32].

References

- [1] N. Zinouri, K. M. Taaffe, D. M. Neyens, Modelling and forecasting daily surgical case volume using time series analysis, *Health Systems* 7 (2) (2018) 111–119.
- [2] T. Wu, K. Xie, D. Xinpin, G. Song, A online boosting approach for traffic flow forecasting under abnormal conditions, in: *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*, IEEE, 2012, pp. 2555–2559.
- [3] J. Cao, Z. Li, J. Li, Financial time series forecasting model based on ceemdan and lstm, *Physica A: Statistical Mechanics and its Applications* 519 (2019) 127–139.

- [4] Z. Lyu, S. Patwardhan, D. Stadem, J. Langfeld, S. Benson, S. Thaelke, T. Desell, Neuroevolution of recurrent neural networks for time series forecasting of coal-fired power plant operating parameters, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion, 2021, pp. 1735–1743.
- [5] T. Fields, G. Hsieh, J. Chenou, Mitigating drift in time series data with noise augmentation, in: 2019 International Conference on Computational Science and Computational Intelligence (CSCI), IEEE, 2019, pp. 227–230.
- [6] D. Sahoo, S. C. Hoi, B. Li, Large scale online multiple kernel regression with application to time-series prediction, *ACM Transactions on Knowledge Discovery from Data (TKDD)* 13 (1) (2019) 1–33.
- [7] D. Nallaperuma, R. Nawaratne, T. Bandaragoda, A. Adikari, S. Nguyen, T. Kempitiya, D. De Silva, D. Alahakoon, D. Pothuhera, Online incremental machine learning platform for big data-driven smart traffic management, *IEEE Transactions on Intelligent Transportation Systems* 20 (12) (2019) 4679–4690.
- [8] S. C. Hoi, D. Sahoo, J. Lu, P. Zhao, Online learning: A comprehensive survey, *Neurocomputing* 459 (2021) 249–289.
- [9] L. Yu, S. Wang, K. K. Lai, An online learning algorithm with adaptive forgetting factors for feedforward neural networks in financial time series forecasting, *Nonlinear dynamics and systems theory* 7 (1) (2007) 51–66.

- [10] R. M. French, Catastrophic forgetting in connectionist networks, *Trends in cognitive sciences* 3 (4) (1999) 128–135.
- [11] J. D. Cryer, *Time series analysis*, Vol. 286, Springer, 1986.
- [12] C. Liu, S. C. Hoi, P. Zhao, J. Sun, Online arima algorithms for time series prediction, in: *Thirtieth AAAI conference on artificial intelligence*, 2016.
- [13] Z. Lyu, A. ElSaid, J. Karns, M. Mkaouer, T. Desell, An experimental study of weight initialization and lamarckian inheritance on neuroevolution, *The 24th International Conference on the Applications of Evolutionary Computation (EvoStar: EvoApps)*.
- [14] Z. Lyu, J. Karnas, A. ElSaid, M. Mkaouer, T. Desell, Improving distributed neuroevolution using island extinction and repopulation, *The 24th International Conference on the Applications of Evolutionary Computation (EvoStar: EvoApps)*.
- [15] Z. Lyu, T. Desell, One-nas: An online neuroevolution based neural architecture search for time series forecasting, *arXiv preprint arXiv:2202.13471*.
- [16] J. D. Cryer, N. Kellet, *Time series analysis*, Springer, 1991.
- [17] R. A. Stine, Estimating properties of autoregressive forecasts, *Journal of the American statistical association* 82 (400) (1987) 1072–1078.
- [18] E. S. Gardner Jr, Exponential smoothing: The state of the art, *Journal of forecasting* 4 (1) (1985) 1–28.

- [19] J. I. Hansen, Triple exponential smoothing; a tool for common stock price prediction.
- [20] B. Siregar, I. Butar-Butar, R. Rahmat, U. Andayani, F. Fahmi, Comparison of exponential smoothing methods in forecasting palm oil real production, in: Journal of Physics: Conference Series, Vol. 801, IOP Publishing, 2017, p. 012004.
- [21] Z. Chen, Q. Xue, R. Xiao, Y. Liu, J. Shen, State of health estimation for lithium-ion batteries based on fusion of autoregressive moving average model and elman neural network, IEEE access 7 (2019) 102662–102678.
- [22] D. Benvenuto, M. Giovanetti, L. Vassallo, S. Angeletti, M. Ciccozzi, Application of the arima model on the covid-2019 epidemic dataset, Data in brief 29 (2020) 105340.
- [23] A. Andreoni, M. N. Postorino, A multivariate arima model to forecast air transport demand, Proceedings of the Association for European Transport and Contributors (2006) 1–14.
- [24] E. V. Tsitsika, C. D. Maravelias, J. Haralabous, Modeling and forecasting pelagic fish production using univariate and multivariate arima models, Fisheries science 73 (2007) 979–988.
- [25] V. Kozitsin, I. Katser, D. Lakontsev, Online forecasting and anomaly detection based on the arima model, Applied Sciences 11 (7) (2021) 3194.
- [26] F. Schmidt, F. Suri-Payer, A. Gulenko, M. Wallschläger, A. Acker, O. Kao, Unsupervised anomaly event detection for cloud monitoring

- using online arima, in: 2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion), IEEE, 2018, pp. 71–76.
- [27] O. Anava, E. Hazan, S. Mannor, O. Shamir, Online learning for time series prediction, in: Conference on learning theory, PMLR, 2013, pp. 172–184.
- [28] M. Han, S. Zhang, M. Xu, T. Qiu, N. Wang, Multivariate chaotic time series online prediction based on improved kernel recursive least squares algorithm, IEEE transactions on cybernetics 49 (4) (2018) 1160–1172.
- [29] O. Anava, E. Hazan, A. Zeevi, Online time series prediction with missing data, in: International Conference on Machine Learning, PMLR, 2015, pp. 2191–2199.
- [30] T. Guo, Z. Xu, X. Yao, H. Chen, K. Aberer, K. Funaya, Robust on-line time series prediction with recurrent neural networks, in: 2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA), Ieee, 2016, pp. 816–825.
- [31] H. Yang, Z. Pan, Q. Tao, Robust and adaptive online time series prediction with long short-term memory, Computational intelligence and neuroscience 2017.
- [32] X. Wang, M. Han, Online sequential extreme learning machine with kernels for nonstationary time series prediction, Neurocomputing 145 (2014) 90–97.

- [33] A. Alsharef, K. Kumar, C. Iwendi, Time series data modeling using advanced machine learning and automl, *Sustainability* 14 (22) (2022) 15292.
- [34] J. Tetteroo, M. Baratchi, H. H. Hoos, Automated machine learning for covid-19 forecasting, *IEEE Access* 10 (2022) 94718–94737.
- [35] B. Celik, P. Singh, J. Vanschoren, Online automl: An adaptive automl framework for online learning, *arXiv preprint arXiv:2201.09750*.
- [36] B. Celik, J. Vanschoren, Adaptation strategies for automated machine learning on evolving data, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43 (9) (2021) 3067–3078.
- [37] J. G. Madrid, H. J. Escalante, E. F. Morales, W.-W. Tu, Y. Yu, L. Sun-Hosoya, I. Guyon, M. Sebag, Towards automl in the presence of drift: first results, *arXiv preprint arXiv:1907.10772*.
- [38] C. Yan, Y. Zhang, Q. Zhang, Y. Yang, X. Jiang, Y. Yang, B. Wang, Privacy-preserving online automl for domain-specific face detection, *arXiv preprint arXiv:2203.08399*.
- [39] I. Ororbia, G. Alexander, F. Linder, J. Snoke, Using neural generative models to release synthetic twitter corpora with reduced stylometric identifiability of users, *arXiv preprint arXiv:1606.01151*.
- [40] L. Cardamone, D. Loiacono, P. L. Lanzi, Learning to drive in the open racing car simulator using online neuroevolution, *IEEE Transactions on Computational Intelligence and AI in Games* 2 (3) (2010) 176–190.

- [41] A. Agogino, K. Stanley, R. Miikkulainen, Online interactive neuro-evolution, *Neural Processing Letters* 11 (1) (2000) 29–38.
- [42] K. O. Stanley, B. D. Bryant, R. Miikkulainen, Real-time neuroevolution in the nero video game, *IEEE transactions on evolutionary computation* 9 (6) (2005) 653–668.
- [43] M. Galassi, N. Capodieci, G. Cabri, L. Leonardi, Evolutionary strategies for novelty-based online neuroevolution in swarm robotics, in: 2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2016, pp. 002026–002032.
- [44] K. Stanley, R. Miikkulainen, Evolving neural networks through augmenting topologies, *Evolutionary computation* 10 (2) (2002) 99–127.
- [45] R. S. Sutton, A. G. Barto, *Reinforcement learning: An introduction*, MIT press, 2018.
- [46] A. Ororbia, A. ElSaid, T. Desell, Investigating recurrent neural network memory structures using neuro-evolution, in: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '19*, ACM, New York, NY, USA, 2019, pp. 446–455. doi:10.1145/3321707.3321795. URL <http://doi.acm.org/10.1145/3321707.3321795>
- [47] A. ElSaid, J. Karns, Z. Lyu, D. Krutz, A. Ororbia, T. Desell, Improving neuroevolutionary transfer learning of deep recurrent neural networks through network-aware adaptation, in: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, 2020, pp. 315–323.

- [48] A. ElSaid, J. Karnas, Z. Lyu, D. Krutz, A. G. Ororbia, T. Desell, Neuro-evolutionary transfer learning through structural adaptation, in: International Conference on the Applications of Evolutionary Computation (Part of EvoStar), Springer, 2020, pp. 610–625.
- [49] A. G. Ororbia II, T. Mikolov, D. Reitter, Learning simpler language models with the differential state framework, *Neural Computation* 0 (0) (2017) 1–26, pMID: 28957029. `arXiv:https://doi.org/10.1162/neco\_a\_01017`, doi:10.1162/neco_a_01017.
URL https://doi.org/10.1162/neco_a_01017
- [50] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, *arXiv preprint arXiv:1412.3555*.
- [51] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Computation* 9 (8) (1997) 1735–1780.
- [52] G.-B. Zhou, J. Wu, C.-L. Zhang, Z.-H. Zhou, Minimal gated unit for recurrent neural networks, *International Journal of Automation and Computing* 13 (3) (2016) 226–234.
- [53] J. Collins, J. Sohl-Dickstein, D. Sussillo, Capacity and trainability in recurrent neural networks, *arXiv preprint arXiv:1611.09913*.
- [54] B. Barker, Message passing interface (mpi), in: Workshop: high performance computing on stampede, Vol. 262, Cornell University Publisher Houston, TX, USA, 2015.

- [55] C. M. Bishop, Training with noise is equivalent to tikhonov regularization, *Neural computation* 7 (1) (1995) 108–116.
- [56] C. M. Bishop, et al., *Neural networks for pattern recognition*, Oxford university press, 1995.
- [57] S. Makridakis, E. Spiliotis, V. Assimakopoulos, Statistical and machine learning forecasting methods: Concerns and ways forward, *PloS one* 13 (3) (2018) e0194889.
- [58] V. K. R. Chimmula, L. Zhang, Time series forecasting of covid-19 transmission in canada using lstm networks, *Chaos, Solitons & Fractals* 135 (2020) 109864.
- [59] R. Fu, Z. Zhang, L. Li, Using lstm and gru neural network methods for traffic flow prediction, in: *2016 31st Youth Academic Annual Conference of Chinese Association of Automation (YAC)*, IEEE, 2016, pp. 324–328.
- [60] O. Marschall, K. Cho, C. Savin, A unified framework of online learning algorithms for training recurrent neural networks, *Journal of machine learning research*.
- [61] F. A. Gers, D. Eck, J. Schmidhuber, Applying lstm to time series predictable through time-window approaches, in: *Neural Nets WIRN Vietri-01*, Springer, 2002, pp. 193–200.

Highlights

Online Evolutionary Neural Architecture Search for Multivariate Non-Stationary Time Series Forecasting

Zimeng Lyu, Alexander Ororbia, Travis Desell

- We present ONE-NAS, the first algorithm capable of designing and training RNNs in real-time as data arrives as an online fashion.
- Our approach uses island and repopulation strategies to preserve diversity and improve performance.
- New genomes are trained with subsets of historical data to prevent data drift, elite candidates are naturally kept in the population.
- ONE-NAS is evaluated on real-world, large scale noisy univariate and multivariate datasets.
- ONE-NAS is asynchronous, distributed and scalable.
- ONE-NAS outperforms classical TSF methods, linear regression models, LSTM and GRU networks trained online, and online ARIMA.

Declaration of interests

☒ The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐ The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: